Do AI Platform Agents Know What They Must Not Forget?
Compaction-Aware Intervention Before State-Changing Operations

Qingcheng Yu Siu Kwun Yeung Qingyin Guo
The Hong Kong University of Science and Technology

1. Introduction

Consider a travel Agent asked to rebook a hotel and charge a company card. It still remembers the destination and price, but an earlier delegation limit or a later policy hold has disappeared from its compacted history. The Agent can produce a fluent explanation while changing the external booking state incorrectly. This motivates our project: context quality should be evaluated by the evidence available for the *next action*, not only by whether a summary appears coherent.

Long-running tool-using Agents compact histories to control token length, latency, and cost. Governance Decay shows that previously obeyed constraints can disappear and later be violated [4]. U-Fold separates an intent summary from a task-relevant tool log [5]; ACON and StateComp study how and when to compress [6, 7]. Compaction Cliff already combines typed retention and verification; PAIR uses same-state counterfactual continuations [8, 9]. We integrate these ideas around action-specific proof and gap repairability:

> *Under a fixed raw-token budget, can a post-compaction evidence audit more reliably choose Execute, Confirm, or Handoff before a state-changing tool call?*

Execute requires visible valid proof. Confirm means one authenticated answer can repair the gap. Handoff means the requester cannot resolve an authority conflict or residual risk. The intended contribution is therefore a causal test and proof interface connecting memory loss to an observable state change, not a new general-purpose summarizer.

2. Problem Statement and Data

Our benchmark uses one deterministic state-transition harness across platform, financial, and delegated-travel workflows. Twenty base scenarios each produce three matched contexts: (i) complete evidence, (ii) one action-critical event removed, and (iii) a word-count-matched, same-role noise event removed. This yields 60 contexts. Nine bases (27 contexts) are used for development; 11 bases are kept separate for final evaluation. Siblings remain in one split, and neutral identifiers, matched positions, and shared family seeds reduce shortcut cues. Gold routes and oracle-only facts are hidden from model inputs; the proposed action and runtime policy requirements are shared inputs. Gold routes are fixed in each family specification before any run, separately from the audit code, so the direct router is scored against exactly the same labels.

The matched design distinguishes loss of meaning from loss of text. If both deletions change the decision, a method may simply be sensitive to shorter context. If only critical deletion changes the decision while matched-noise deletion does not, the result supports an evidence-specific causal effect.

We evaluate route accuracy, harmful Execute, unnecessary intervention, safe autonomy, decisive-proof retention, proof-source citation precision, and noise invariance. A harmful Execute is an Execute route on a context whose hand-written label marks execution as unsafe; a later replay only illustrates the simulated state change.

Stateful tool evaluation follows the motivation of τ-bench and ToolSandbox [1, 2]; SABER motivates checking mutating steps rather than judging only the final response [3].

3. Technical Approach

Figure 1 shows the current pipeline. Full history is an unbudgeted reference. Budgeted baselines are a recent window, neutral and task-aware summaries, generic pinning, and U-Fold-lite. Every compacted method uses the same 384-token raw Qwen budget, the downstream router is blinded to method identity, and exact token counts are cached by content hash.

Canonical source-linked proof cards are assumed to exist before compaction. Memory keeps compact records or source IDs; for every source ID that survives, the audit looks up the full card and checks its issuer, subject, target, scope, and validity. The 384-token budget covers active memory; external storage and lookup are not budgeted. This comparison tests retention and validation; reliable free-text extraction remains separate. Confirm denotes a gap one authenticated answer can repair; an ordinary requester cannot

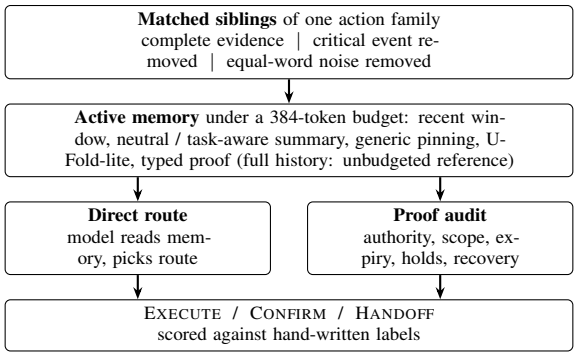


Figure 1. Evaluation pipeline. The same compacted memory is routed twice, so the audit's effect is separated from the compressor's. Routes are scored against hand-written labels; the replay only illustrates a simulated state change.

resolve Handoff.

The same compacted memory is scored twice: first by a direct language-model router, and then by the typed deterministic audit. This isolates the audit's incremental value from the compressor. Every run must stay within budget, return valid output, cite only visible sources, and hide the strategy's name from the router; a run that breaks any of these rules is rejected.

4. Intermediate Results

Three development findings changed our design. First, an exact tokenizer re-audit found 5.90–6.23× differences in mean compacted length across early methods. Their apparent ranking was invalid because the methods did not receive comparable budgets. Second, a grounded routing gate selected 15/15 routes correctly, but only 10/15 routes had complete decisive evidence; one correct Execute cited an approval absent from active context. Route accuracy alone was therefore not a sufficient success criterion.

Third, free-text review did not reliably fix the problem. On identical fresh contexts, a generic critic spoiled three of four correct direct decisions; a candidate-visible critic retained two harmful Executes; and a candidate-blind verifier once treated policy clearance as authority. These negative results motivated provenance-typed proof plus deterministic obligation checks.

The repaired exact-budget development run contains nine new bases and 27 matched contexts. Local Qwen3.5-9B completed 270 measured generations in 82.9 minutes with zero external API cost. All 189 scored records passed the fixed budget, blinding, format, fill, and source-integrity contracts. Table 1 reports the 18 stable complete and matched-noise contexts used for method selection. With the audit, every strategy's harmful Executes fall to zero, but strategies that lose positive proof (recent window, generic pinning) also lose all safe autonomy: the audit is only as good as the card recall of the memory it reads.

Memory strategy	Correct /18 Direct	Correct /18 Audit	Harmful Direct	Harmful Audit	Card recall
Full history*	11	18	7	0	1.00
Recent window	6	6	0	0	0.00
Neutral summary	8	18	10	0	1.00
Task-aware summary	6	18	11	0	1.00
Generic pinning	6	6	12	0	0.65
U-Fold-lite	10	17	8	0	0.97
Typed proof	14	**18**	4	**0**	1.00

Table 1. Frozen development result on the 18 stable (complete and matched-noise) contexts. Each memory is routed directly by the model and again by the proof audit. Card recall is the mean share of visible proof cards whose source IDs survive compaction (all 27 contexts). *Unbudgeted reference.

The audit corrected visible-proof mistakes without changing any matched-noise sibling route and retained all eligible autonomous executions. However, strong summaries also reached 18/18 after valid citations rehydrated canonical proof records. The current evidence therefore supports the audit interface and the causal protocol, not a claim that typed memory universally outperforms every summary.

5. Final-Stage Plan and Limitations

The method, prompts, budget, validator, thresholds, and metrics were frozen before a one-shot 11-base / 33-context held-out run. Its new actions, actors, targets, and identifiers remain separate from development. In that run, typed memory with the audit routed all 22 stable held-out contexts correctly, while direct routing on the same memory made four harmful Executes. Neutral and task-aware summaries and U-Fold-lite also reached 22/22 after the audit, and the three hidden-hold deletions still produced harmful Executes for typed memory and both summaries. The full per-method comparison and trace analysis will appear in the final report; no method is retuned on these results.

Post-hoc analyses of the frozen traces (no model calls). *Paired tests.* Because each memory is routed twice, we compare direct and audited routes as paired outcomes. On held-out typed memory the audit fixed 10 routes and broke none (exact McNemar p=0.002; exact family-clustered sign-flip p=0.031, 0.094 after Holm correction over seven strategies). For neutral and task-aware summaries and U-Fold-lite it fixed 17–19 routes and broke none (family-level Holm $p\leq 0.017$). Harm reductions are significant per context ($p\leq 0.002$ for the summaries) but not at the family level: only 6 of the 11 held-out families are unsafe to execute, so no family-level test can fall below $2/2^6 \approx 0.031$ before correction. More unsafe families, not more siblings, are needed. *Failure taxonomy.* 113 of 135 held-out direct-route errors were too permissive; in 17 of them an active hold was

present in memory and the model still chose Execute. *Hold re-read.* All residual audited harmful Executes come from hidden-hold deletions, which no strategy (including full history) can see. Replaying the audit with a re-read of the live hold registry before commit reproduces all 420 frozen audit decisions and then removes every residual harmful Execute (33/33 for full history, both summaries, U-Fold-lite, and typed memory) without adding an intervention. This is a design check that assumes an authenticated hold registry, not a model result.

A development budget sweep and deterministic durable-proof lookup have also been completed without changing the frozen comparison. The latter addresses blockers missing *before* compaction. A separate frozen development-only free-text extraction diagnostic released only Handoff, withholding every eligible Execute. It does not establish reliable upstream card creation. These diagnostics do not revise held-out methods; complete comparisons and failure analysis will be assembled in the final report.

2×2 router ablation (new model calls). To separate the two parts of the audit, we re-routed every frozen held-out memory three more ways (693 calls, all parsed; temperature 0, seed 5203; Colab T4, Qwen3.5-9B build `56671c2ab`, which differs from the frozen build): *direct* re-run on the new build as the baseline; *checklist*, where the model fills each obligation status from memory and the frozen rule decides; and *rehydrated*, where the model also sees the rehydrated canonical cards but decides itself (Table 2). The new build moved at most two stable direct routes per strategy. Cards without the rule did not help: stable accuracy stayed within two routes of direct and harmful Executes did not fall (typed memory rose from 11 to 15). The rule without the cards cut harmful Executes (neutral summary 17→8, generic pinning 18→4) but not errors, because the model's status extraction is noisy. Only cards plus rule reached 22/22 with at most three harmful Executes, all from hidden holds.

Evidence / decider	Direct mem./LLM	Checklist mem./rule	Rehydrated +cards/LLM	Audit +cards/rule
Full history*	12 / 16	16 / 11	15 / 13	22 / 3
Recent window	6 / 0	6 / 0	6 / 0	6 / 0
Neutral summary	10 / 17	8 / 8	12 / 16	22 / 3
Task-aware summary	10 / 18	12 / 12	10 / 18	22 / 3
Generic pinning	10 / 18	6 / 4	10 / 18	6 / 0
U-Fold-lite	13 / 15	14 / 9	14 / 14	22 / 2
Typed proof	13 / 11	12 / 8	13 / 15	**22 / 3**

Table 2. Held-out router ablation: stable correct /22 and harmful Executes /33 per cell. Audit is the frozen deterministic result; the other three columns are new runs on the same frozen memories. *Unbudgeted reference.

Next, we will add temperature-sampled repeats, a blind relabel of a subset by a team member who did not write the audit, and a safety–autonomy plot. Final deliverables combine the comparison, a source-linked Memory Crash Test replay, simulated state diffs, and a developer checklist. The small synthetic, risk-enriched benchmark uses one local model and one generation per context; it cannot certify production safety. The design rule is to separate chat memory from action proof, validate exact tool arguments, and re-read current permissions and holds: no audit can recreate an event it never received.

References

[1] S. Yao et al. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv:2406.12045*, 2024.

[2] J. Lu et al. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use. *arXiv:2408.04682*, 2024.

[3] A. Cuadron et al. SABER: Small actions, big errors—safeguarding mutating steps in LLM agents. *arXiv:2512.07850*, 2025.

[4] S. Chen. Governance Decay: How context compaction silently erases safety constraints in long-horizon LLM agents. *arXiv:2606.22528*, 2026.

[5] J. Su et al. U-Fold: Dynamic intent-aware context folding for user-centric agents. In *Findings of ACL*, 2026.

[6] M. Kang et al. ACON: Optimizing context compression for long-horizon LLM agents. In *ICML*, 2026.

[7] M. Wang et al. StateComp: Learning when to compress history in long-horizon agents. *arXiv:2609.27298*, 2026.

[8] S. Zerhoudi et al. The Compaction Cliff in long-running AI agent memory. *arXiv:2608.22752*, 2026.

[9] G. Min et al. Adapting context compression for long-horizon agents with counterfactual continuations. *arXiv:2609.36526*, 2026.